

Revolut Java Graduate Programme Technical Conversation Interview

Prepared for **Wojciech** · 2027 intake, Poland (Kraków) · 19 September 2026

FORMAT

60 min

Google Meet, your own IDE, JUnit and Mockito, no Spring, no AI tools

CODING

3 to 4 stories

Pasted into the chat one at a time; each finished and tested before the next

THE TRAP

Lock order

Two-account transfer under many threads, then deep database Q&A

What this pack is

Everything in this document comes from Revolut's own published guidance and from candidate reports dated 2024 to September 2026 (PracHub, Taro, Reddit, 4programmers.net, GitHub). Sources are listed at the end.

It gives you the format, the exact task families candidates were given, a reference Java solution with the concurrency test that proves it, the database questions with model answers, and a practice repository of six ready-to-run projects with solution branches and a day-by-day schedule: github.com/amigoscode/interview-prep

- | | | | |
|---|---|----|--|
| 1 | The one-page summary | 6 | The technical conversation: questions and model answers |
| 2 | Where this round sits in the process | 7 | The practice repo: projects to build between now and the interview |
| 3 | What candidates actually got | 8 | Interview-day script |
| 4 | The main task: in-memory ledger with safe transfers | 9 | Poland-specific notes |
| 5 | Fallback tasks (first-round style) | 10 | Sources |

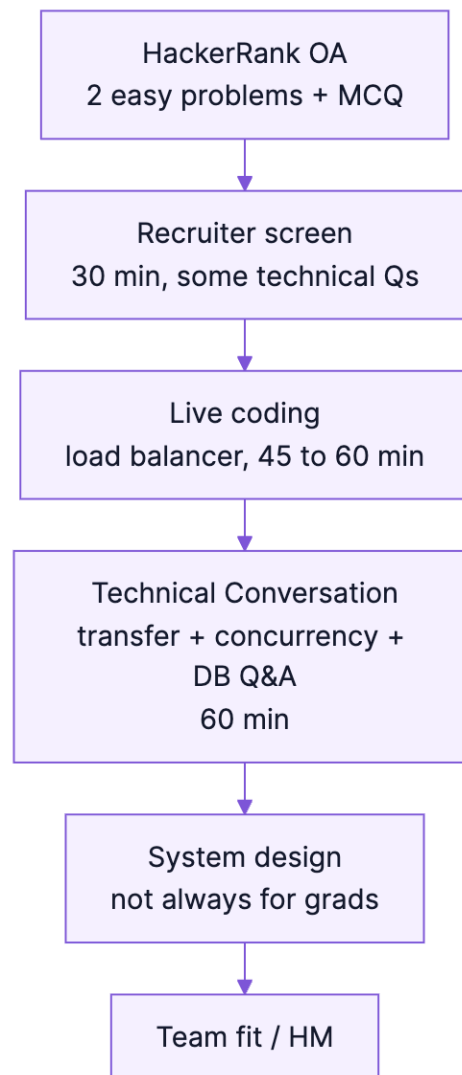
1. The one-page summary

WHAT	EXPECT
Length	60 minutes on Google Meet, camera on, screen share of your own IDE
Split	About 35 to 40 min coding, 20 min technical conversation
Setup	Empty Java 17/21 project you created beforehand with JUnit 5, Mockito (AssertJ optional). No Spring. No AI tools. IDE autocomplete is allowed
Task delivery	Interviewer pastes a skeleton and 3 to 4 "user stories" into the Meet chat, one at a time. Finish each with tests before the next is revealed
The task	Almost certainly an in-memory bank ledger: accounts, deposit, withdraw, transfer, then "make it thread-safe". Fallbacks seen: load balancer, URL shortener
The trap	Story 3 or 4 adds concurrency. Locking two accounts in the wrong order deadlocks. <code>synchronized</code> on the service method is marked down. Check-then-act outside the lock is a race
Q&A	Deep database theory: isolation levels, optimistic vs pessimistic locking, <code>SELECT FOR UPDATE</code> , indexes, replication, sharding, migrations. Plus DDD, CQRS, event sourcing, stability patterns, "how would you investigate a slow service"
Graded on	Clarifying before coding, working MVP per story, tests with edge cases, SOLID and naming, thread safety, talking continuously
Outcome	Feedback within 1 to 2 days, blunt, checklist-style. Poland pass rate reported around 20%

The three things that get graduates rejected in this round, in order of frequency:

1. A deadlock or race in the transfer function (Aug 2026 New Grad rejection: "wrote a deadlock using a read-write lock and only caught it after the interviewer pointed it out").
2. Vague database answers ("What isolation level fixes your transfer?" with no concrete answer).
3. Not knowing the words: DDD, CQRS, "buckets" when asked how HashSet checks duplicates (UK grad, Dec 2024, rejected for saying "memory addresses").

2. Where this round sits in the process



The email Wojciech received names the fourth box exactly, and its prep bullets ("concurrency problems and how to solve them on database levels") match every report of that round. Some graduate candidates report the two coding rounds merged into one "live coding + technical conversation" session, so prepare for both task families, with the ledger weighted highest.

What Revolut says about it in their own words (blog, March 2025):

"This stage delves deeper into your technical expertise, covering topics such as concurrency, databases, and system architecture. It's not just about what you know but how you apply that knowledge. You'll encounter a blend of conceptual questions and hands-on problem-solving tasks."

Concurrency: synchronisation, thread safety, parallel processing. Databases: indexing, query performance, transaction handling, distributed systems. Architecture: microservices, DDD, event-driven.

3. What candidates actually got

SOURCE, DATE	TASK	Q&A AFTERWARDS	RESULT
PracHub, New Grad Java EU, Aug 2026	V01: load balancer, fixed size, random pick. V02: transfer function "with the concurrency angle"	How Revolut keeps DBs in sync across regions, how the DB handles concurrency, how to investigate a slow service, what is DDD, what is CQRS	Rejected next day (deadlock with RW lock, didn't know DDD/CQRS)
PracHub, SWE, Sept 2026	Correct concurrent logic, safe lock handling, real-system edge cases	Transaction management, locking, CQRS, sharding, indexes. Even the HR round had these	n/a
Taro, Java Dev Poland, Aug 2024	Round 1: load balancer MVP then extra requirements. Round 2: money transfer service with proper locking for multithreading, plus find-the-bug in given snippets	Isolation levels and how implemented, DB lock types, DB scaling options with pros/cons, migrations, deployment strategies, microservices, stability patterns	Rejected
Taro, SWE Poland, July 2025	Round 1: live coding, TDD, SOLID. Round 2: "financial domain and multithreading, covering both code and SQL", plus DDD and scalability	HashMap read complexity	Offer
Taro, Java Backend Porto	Round 2 (called "Technical Conversation Interview"): transfer money between accounts, "interviewer primarily wanted locks on both account objects"	Query optimisation, indexes, optimistic/pessimistic concurrency, isolation levels, partitions, scaling, streams, event sourcing, CQRS, fault tolerance	Rejected ("answers too vague", struggled with double object lock)
Taro, Graduate SWE UK, Dec 2024	Load balancer returning a service with requirements	HashSet contains complexity, how HashSet checks equality	Rejected (didn't say "buckets")
Reddit, Senior Java, Feb 2026 (recruiter's own brief)	3 tasks, own IDE with JUnit/Mockito, each fully tested before the next, "very concurrency heavy, has a concurrency trap", marked on SOLID and DDD, TDD optional	Commenter: "look up why you'd order locks when making payments between two accounts, and why synchronizing the method is a bad idea"	Passed to system design
Reddit, Java backend, June 2026	"4 tasks as user stories, need to complete all 4 plus how to make it concurrent. Skeleton given via the Meet chat. No JDBC needed. Talk as much as you can throughout"		Passed

SOURCE, DATE	TASK	Q&A AFTERWARDS	RESULT
4programmers.net, Kraków	Account transfer app, no Spring; "no concurrency handling" was the single-line rejection. Another: second stage "almost 2 hours, only database questions"		

4. The main task: in-memory ledger with safe transfers

4.1 The likely user-story sequence

1. **Accounts and balance.** Create an account with an id and opening balance. Deposit. Withdraw. Get balance. Reject unknown account, negative or zero amounts, withdrawal beyond balance.
2. **Transfer.** Move an amount from A to B atomically. Reject same-account transfer, insufficient funds. Either both sides change or neither does.
3. **Concurrency.** "Many threads will call transfer at the same time. Make it safe." Prove it with a test.
4. **An extension.** Seen variants: transaction history per account, idempotency key on transfer (same request twice must not double-charge), daily withdrawal limit, a `TransferResult` with a reason code instead of exceptions.

4.2 The trap, and the answers the interviewer wants to hear

MISTAKE	WHY IT FAILS	WHAT TO DO INSTEAD
<code>synchronized</code> on the whole service method	Correct but serialises the entire bank; interviewer will ask "and if we have 10 million accounts?"	Lock per account
Lock A then B in the order the caller passed them	Thread 1 does A then B, thread 2 does B then A: deadlock	Always acquire in a global order (by account id). Or <code>tryLock</code> with timeout and back off
Check balance, then lock, then debit	Two threads both pass the check, both debit: overdraft	Check and debit inside the same critical section
<code>ReadWriteLock</code> for a transfer	A transfer is a write on both sides; a read lock upgraded to write is exactly how the Aug 2026 grad deadlocked	Plain <code>ReentrantLock</code> per account, or <code>synchronized(account)</code>
<code>double</code> for money	Rounding	<code>BigDecimal</code> with scale 2
<code>HashMap</code> for the account store	Not thread-safe under concurrent put	<code>ConcurrentHashMap</code> , <code>computeIfAbsent</code> for creation
No concurrency test	"No concurrency handling" was a rejection on its own in Kraków	Write the N-threads test below and show total money is conserved

Be ready to also explain the alternative you did **not** pick: a lock-free approach with `AtomicReference<BigDecimal>` and compare-and-set loops per account works for deposit/withdraw but does not give you atomic two-account transfer without a lock or a two-phase protocol, which is why the lock with ordering is the right answer at this size.

4.3 Reference solution (Java 21, no frameworks)

Build this from memory at least 4 times under a 35 minute timer before the interview. Type it, do not paste it.

```

package ledger;

import java.math.BigDecimal;
import java.util.concurrent.locks.ReentrantLock;

public final class Account {
    private final String id;
    private BigDecimal balance;           // guarded by lock
    final ReentrantLock lock = new ReentrantLock();

    Account(String id, BigDecimal opening) {
        this.id = id;
        this.balance = opening;
    }

    String id() { return id; }

    BigDecimal balance() {
        lock.lock();
        try { return balance; } finally { lock.unlock(); }
    }

    // callers must hold lock
    void credit(BigDecimal amount) { balance = balance.add(amount); }
    void debit(BigDecimal amount) { balance = balance.subtract(amount); }
}

```

```

package ledger;

import java.math.BigDecimal;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

public final class Ledger {
    private final Map<String, Account> accounts = new ConcurrentHashMap<>();

    public void open(String id, BigDecimal opening) {
        requirePositiveOrZero(opening);
        if (accounts.putIfAbsent(id, new Account(id, opening)) != null) {
            throw new IllegalArgumentException("account exists: " + id);
        }
    }

    public BigDecimal balance(String id) {
        return find(id).balance();
    }

    public void deposit(String id, BigDecimal amount) {
        requirePositive(amount);
        Account a = find(id);
        a.lock.lock();
        try { a.credit(amount); } finally { a.lock.unlock(); }
    }

    public void withdraw(String id, BigDecimal amount) {
        requirePositive(amount);
        Account a = find(id);
        a.lock.lock();
        try {
            if (a.balance().compareTo(amount) < 0) {

```

```

        throw new InsufficientFundsException(id);
    }
    a.debit(amount);
} finally { a.lock.unlock(); }
}

public void transfer(String fromId, String toId, BigDecimal amount) {
    requirePositive(amount);
    if (fromId.equals(toId)) {
        throw new IllegalArgumentException("cannot transfer to same account");
    }
    Account from = find(fromId);
    Account to = find(toId);

    // Global lock order by id prevents the A→B / B→A deadlock.
    Account first = from.id().compareTo(to.id()) < 0 ? from : to;
    Account second = first == from ? to : from;

    first.lock.lock();
    try {
        second.lock.lock();
        try {
            if (from.balance().compareTo(amount) < 0) { // check-then-act inside the locks
                throw new InsufficientFundsException(fromId);
            }
            from.debit(amount);
            to.credit(amount);
        } finally { second.lock.unlock(); }
    } finally { first.lock.unlock(); }
}

private Account find(String id) {
    Account a = accounts.get(id);
    if (a == null) throw new AccountNotFoundException(id);
    return a;
}

private static void requirePositive(BigDecimal amount) {
    if (amount == null || amount.signum() ≤ 0) {
        throw new IllegalArgumentException("amount must be positive");
    }
}

private static void requirePositiveOrZero(BigDecimal amount) {
    if (amount == null || amount.signum() < 0) {
        throw new IllegalArgumentException("amount must not be negative");
    }
}
}

```

Note: `Account.balance()` takes the lock again inside `transfer`. That is fine because `ReentrantLock` is reentrant, and saying that sentence out loud is worth a point. If asked to simplify, expose a package-private `rawBalance()` for use under the lock.

```

package ledger;

public class InsufficientFundsException extends RuntimeException {
    public InsufficientFundsException(String id) { super("insufficient funds in " + id); }
}

public class AccountNotFoundException extends RuntimeException {

```

```

    public AccountNotFoundException(String id) { super("no such account: " + id); }
}

```

4.4 Tests (JUnit 5)

```

package ledger;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import java.math.BigDecimal;
import java.util.concurrent.*;
import static org.junit.jupiter.api.Assertions.*;

class LedgerTest {
    Ledger ledger;

    @BeforeEach
    void setUp() {
        ledger = new Ledger();
        ledger.open("A", new BigDecimal("100.00"));
        ledger.open("B", new BigDecimal("50.00"));
    }

    @Test void transferMovesMoney() {
        ledger.transfer("A", "B", new BigDecimal("30.00"));
        assertEquals(new BigDecimal("70.00"), ledger.balance("A"));
        assertEquals(new BigDecimal("80.00"), ledger.balance("B"));
    }

    @Test void transferRejectsInsufficientFunds() {
        assertThrows(InsufficientFundsException.class,
            () → ledger.transfer("A", "B", new BigDecimal("100.01")));
        assertEquals(new BigDecimal("100.00"), ledger.balance("A")); // nothing changed
    }

    @Test void transferRejectsSameAccount() {
        assertThrows(IllegalArgumentException.class,
            () → ledger.transfer("A", "A", BigDecimal.ONE));
    }

    @Test void transferRejectsNonPositiveAmount() {
        assertThrows(IllegalArgumentException.class,
            () → ledger.transfer("A", "B", BigDecimal.ZERO));
        assertThrows(IllegalArgumentException.class,
            () → ledger.transfer("A", "B", new BigDecimal("-1")));
    }

    @Test void transferRejectsUnknownAccount() {
        assertThrows(AccountNotFoundException.class,
            () → ledger.transfer("A", "Z", BigDecimal.ONE));
    }

    @Test void concurrentTransfersConserveTotalAndDoNotDeadlock() throws Exception {
        int threads = 16, perThread = 1_000;
        ExecutorService pool = Executors.newFixedThreadPool(threads);
        CountDownLatch start = new CountDownLatch(1);
        CountDownLatch done = new CountDownLatch(threads);

        for (int t = 0; t < threads; t++) {
            boolean forward = t % 2 == 0; // half go A→B, half go B→A

```

```

        pool.submit(() → {
            start.await();
            for (int i = 0; i < perThread; i++) {
                try {
                    if (forward) ledger.transfer("A", "B", BigDecimal.ONE);
                    else        ledger.transfer("B", "A", BigDecimal.ONE);
                } catch (InsufficientFundsException ignored) { }
            }
            done.countDown();
            return null;
        });
    }
    start.countDown();
    assertTrue(done.await(10, TimeUnit.SECONDS), "deadlock or too slow");
    pool.shutdownNow();

    BigDecimal total = ledger.balance("A").add(ledger.balance("B"));
    assertEquals(new BigDecimal("150.00"), total);
    assertTrue(ledger.balance("A").signum() ≥ 0);
    assertTrue(ledger.balance("B").signum() ≥ 0);
}
}

```

The last test is the one that matters. Before you write the lock ordering, run it with locks taken in caller order and watch it hang; that gives you a story to tell ("here is the deadlock, here is the fix").

4.5 Story 4 extensions, so you are not surprised

- **Idempotency key:** `transfer(key, from, to, amount)`. Keep a `ConcurrentHashMap<String, TransferResult>`; `computeIfAbsent(key, k → doTransfer(...))` makes the replay return the first result without re-charging. Say out loud that in production this map lives in Postgres or Redis with a TTL.
- **Transaction history:** append an immutable `Transaction` record to a `CopyOnWriteArrayList` or a synchronized list per account, inside the same lock as the balance change, so history and balance never disagree.
- **Daily limit:** track `withdrawnToday` on the account under the same lock; reset by date. Mention the clock should be injected (`clock`) for testability.
- **Result object instead of exceptions:** a sealed interface `TransferResult { Success, InsufficientFunds, AccountNotFound }` and a `switch` pattern match; shows Java 21 fluency.

5. Fallback tasks (first-round style)

5.1 Load balancer

Stories seen: (1) register an instance (unique, max N, return boolean), (2) unregister, (3) `get()` returns a random instance, throws if none, (4) add round robin via a strategy, (5) make it thread-safe.

```
public interface SelectionStrategy { String select(List<String> instances); }

public final class RoundRobin implements SelectionStrategy {
    private final AtomicInteger next = new AtomicInteger();
    public String select(List<String> instances) {
        if (instances.isEmpty()) throw new NoInstancesException();
        int i = Math.floorMod(next.getAndIncrement(), instances.size());
        return instances.get(i);
    }
}

public final class LoadBalancer {
    private final int capacity;
    private final Set<String> instances = new LinkedHashSet<>(); // guarded by this
    private final SelectionStrategy strategy;

    public LoadBalancer(int capacity, SelectionStrategy strategy) {
        if (capacity ≤ 0) throw new IllegalArgumentException("capacity must be positive");
        this.capacity = capacity; this.strategy = strategy;
    }

    public synchronized boolean register(String instance) {
        if (instances.size() ≥ capacity) return false;
        return instances.add(instance);
    }

    public synchronized boolean unregister(String instance) { return instances.remove(instance); }
    public synchronized String get() { return strategy.select(List.copyOf(instances)); }
}
```

Expect follow-ups: why `LinkedHashSet` (uniqueness plus stable order for round robin), what is the complexity of `contains` ($O(1)$ average, hash to a **bucket**, then `equals` within the bucket), what if two instances are equal by address but different objects (`equals` / `hashCode` contract), is `synchronized` acceptable here (yes, tiny critical section, low contention; mention `ReadWriteLock` or `CopyOnWriteArrayList` as the read-heavy alternative).

5.2 URL shortener

Stories: `shorten(long)` returns a short code, same long URL returns the same code, `expand(code)` returns the long URL or empty, validate URL, pluggable code strategy (counter + base62, random, hash), thread-safe. Two `ConcurrentHashMap`s and an `AtomicLong` counter cover it; the trap is making `shorten` idempotent under concurrency (`computeIfAbsent`).

5.3 Rate limiter

Occasionally asked. Token bucket per client key: `tryAcquire(key)` returns boolean, refill by elapsed time, injected `Clock`. Thread-safe via `ConcurrentHashMap.compute`.

6. The technical conversation: questions and model answers

Reports describe this half as "vague questions, in-depth answers expected". Answer in the shape: definition, the problem it solves, a concrete example (preferably the transfer you just wrote), trade-off.

6.1 Transactions and isolation

ACID. Atomicity (all or nothing, via WAL/undo), Consistency (constraints hold), Isolation (concurrent transactions behave as if serial, to a configurable degree), Durability (committed means on disk, via write-ahead log and fsync).

Isolation levels and the anomalies they allow (Postgres):

LEVEL	DIRTY READ	NON-REPEATABLE READ	PHANTOM READ	LOST UPDATE / WRITE SKEW
Read uncommitted	Postgres treats as Read committed			
Read committed (Postgres default)	No	Yes	Yes	Yes
Repeatable read (snapshot)	No	No	No in Postgres	Lost update prevented (serialization error), write skew possible
Serializable (SSI)	No	No	No	No

"Which level fixes your transfer?" With plain `SELECT` then `UPDATE` at Read committed, two transfers can both read balance 100 and both debit 80: lost update. Fixes, in the order you should give them: (1) `SELECT ... FOR UPDATE` on both rows in id order (pessimistic, what Revolut reportedly prefers), (2) a single conditional `UPDATE accounts SET balance = balance - 80 WHERE id = ? AND balance ≥ 80` and check the row count, (3) optimistic version column, (4) Serializable and retry on `40001`. Repeatable read alone raises an error on the second writer in Postgres, which you must retry.

6.2 Locking

Pessimistic: take the lock before reading. Row locks (`FOR UPDATE`, `FOR SHARE`, `FOR NO KEY UPDATE`), table locks, advisory locks. Safe under high contention, risks deadlock (Postgres detects and aborts one), holds locks for the transaction duration so keep transactions short. Lock rows in a consistent order, same rule as in Java.

Optimistic: read with a version, write with `WHERE version = ?`, if 0 rows updated then someone else won: retry or fail. No locks held between read and write, great for low contention, wasteful under high contention. `SELECT ... FOR UPDATE SKIP LOCKED` is the queue-worker trick.

Externalised: Redis `SET NX PX` or a lock table, when the critical section spans services. Mention fencing tokens.

```

-- Pessimistic transfer, id-ordered
BEGIN;
SELECT balance FROM accounts WHERE id IN (1, 2) ORDER BY id FOR UPDATE;
UPDATE accounts SET balance = balance - 100 WHERE id = 1 AND balance ≥ 100;
-- check row count = 1 else ROLLBACK
UPDATE accounts SET balance = balance + 100 WHERE id = 2;
COMMIT;

-- Optimistic transfer
BEGIN;
UPDATE accounts SET balance = balance - 100, version = version + 1
  WHERE id = 1 AND version = :v1 AND balance ≥ 100;      -- expect 1 row
UPDATE accounts SET balance = balance + 100, version = version + 1
  WHERE id = 2 AND version = :v2;                        -- expect 1 row
COMMIT;                                                  -- else ROLLBACK and retry

```

6.3 Indexes and query performance

B-tree default; composite index column order matters (leftmost prefix); covering index avoids heap fetch; partial index for `WHERE status = 'PENDING'`; indexes cost writes and space. Read `EXPLAIN ANALYZE`: Seq Scan vs Index Scan, estimated vs actual rows. N+1 and how to spot it. Why `SELECT *` hurts. Why a function on an indexed column (`lower(email)`) disables the index unless the index is on the expression.

6.4 Scaling and distributed data

Vertical first. Read replicas (async replication, replica lag, read-your-writes problem). Sharding by customer id (cross-shard transfer needs saga or two-phase commit; Revolut asked "how does a global bank keep databases in sync across regions": answer with primary region per customer, async replication, idempotent events, eventual consistency for non-critical reads). Partitioning a big table by date. Caching with Redis and the invalidation problem. CAP in one sentence: under a partition choose consistency or availability; a ledger chooses consistency.

6.5 Migrations, deployments, stability

Migrations with Flyway (Revolut uses it): expand and contract (add nullable column, dual write, backfill, switch reads, drop old), never rename in one step, adding an index `CONCURRENTLY`. Rollback is a forward migration, not a restore. Deployments: rolling, blue/green, canary, feature flags. Stability patterns: timeouts, retries with jitter and idempotency, circuit breaker, bulkhead, rate limiting, dead-letter queue.

6.6 Architecture vocabulary they expect a grad to define

- **DDD**: entity (identity), value object (immutable, equality by value, e.g. `Money`), aggregate (consistency boundary, e.g. `Account`), repository, domain event, bounded context. Say your `Ledger` is an application service and `Account` is the aggregate.
- **CQRS**: separate write model from read model; commands change state, queries read a projection.
- **Event sourcing**: store the events (`Deposited`, `Transferred`), derive balance by replay; audit for free, needs snapshots.

- **Microservices:** one bounded context per service, own database, communicate by API or events, the outbox pattern for reliable events.
- **Observability:** logs, metrics, traces; "user says it is slow": check dashboards for latency percentiles and error rate, find the slow span in a trace, look at DB `pg_stat_activity` for locks and slow queries, then the recent deploy.

6.7 Java concurrency and core Java

- `synchronized` vs `ReentrantLock` (`tryLock`, fairness, interruptible, conditions). `volatile` gives visibility not atomicity. Happens-before. `Atomic*` and CAS. `ConcurrentHashMap` vs `Collections.synchronizedMap`. `ExecutorService`, `CompletableFuture`, virtual threads in Java 21 (one sentence: cheap blocking threads, still need the same locks).
- Deadlock (circular wait; fix by ordering or timeout), livelock, starvation. The four Coffman conditions.
- `HashMap` internals: array of buckets, hash spread, chaining to a treeified bin above 8, load factor 0.75 and resize, `equals` / `hashCode` contract. `HashSet.contains` is $O(1)$ average, $O(\log n)$ worst in a treeified bin.
- `Optional`, records, sealed interfaces, pattern matching in `switch`, streams. Immutability. Why `BigDecimal` for money and `compareTo` not `equals` for it.

7. The practice repo: projects to build between now and the interview

Every project below is a ready-made module in Amigoscode's interview practice repository, under `java/live-coding/revolut/`, written the way the interviewer gives it: user stories revealed one at a time, a compiling skeleton, a starter test, and the questions asked afterwards. Nothing to install beyond a JDK: the Maven wrapper fetches Maven, the database module runs on in-memory H2, and there is no Spring anywhere, which is the interview's own setup.

Repository: github.com/amigoscode/interview-prep

7.1 Getting started (10 minutes)

```
git clone https://github.com/amigoscode/interview-prep.git
cd interview-prep/java/live-coding/revolut
./mvnw test                # every module builds, starter tests are green
./mvnw -pl 01-ledger test  # one module
```

All `./mvnw` commands run from `java/live-coding/revolut/`. Windows: `mvnw.cmd` instead of `./mvnw`. IntelliJ: File, Open, choose `java/live-coding/revolut/pom.xml`, run any test class with Ctrl+Shift+F10. Requirements: JDK 21 or newer on the PATH. Turn off Copilot and every AI assistant in the IDE; the interview forbids them and you want to rehearse the way you will play.

Work on your own branch so the skeletons stay clean: `git checkout -b wojciech/revolut-01-ledger main`. To reset a module to the skeleton: `git checkout main -- java/live-coding/revolut/01-ledger`.

7.2 How to practise a module

1. Open the module's `README.md` but read **only the next story**. Cover the rest.
2. Start a timer. Say the story back in one sentence, ask two clarifying questions out loud and write your answers down as assumptions, state the plan.
3. Remove `@Disabled` from the starter test, watch it fail, make it pass, add the edge cases, run the module green.
4. Say "done", reveal the next story. Do not refactor mid-story; note it, come back at the end.
5. When the timer ends, stop and answer the "questions the interviewer asks afterwards" out loud, in full sentences.
6. Then, and only then, compare with the solution branch.

7.3 Solutions

Each module has a complete, tested solution on its own branch:

```
git checkout solutions/revolut/01-ledger          # then ./mvnw -pl 01-ledger test
git checkout solutions/revolut/02-load-balancer
git checkout solutions/revolut/03-url-shortener
git checkout solutions/revolut/04-rate-limiter
git checkout solutions/revolut/05-sql-transfer
git checkout solutions/revolut/06-ledger-http      # includes the 01-ledger solution it depends on
```

```
git checkout main # back to the skeletons
git diff main solutions/revolut/01-ledger -- 01-ledger # exactly what the solution added
```

7.4 The modules

FOLDER	REHEARSES	TIME BOX	DO IT
01-ledger	The interview task: accounts, transfer, thread-safe transfer with lock ordering, then one extension (idempotency key, history, daily limit, result type)	35 min	4 to 5 times
02-load-balancer	The first-round task: register/unregister/get, random and round-robin strategies, thread safety	30 min	twice
03-url-shortener	The other first-round task; <code>computeIfAbsent</code> , base62, idempotency under 20 threads	30 min	once
04-rate-limiter	Token bucket with an injected <code>Clock</code> ; time-dependent code under test	30 min	once
05-sql-transfer	The same transfer in SQL on H2: reproduce the lost update, fix it with <code>FOR UPDATE</code> , then optimistic locking, then one-statement	one evening	once, take notes
06-ledger-http	The ledger behind a Spring-free Javalin API; Mockito for the handlers; <code>Idempotency-Key</code> header	one evening	optional

01-ledger is the one to repeat. Stories: accounts; transfer; many threads; one extension. The solution branch's `LedgerConcurrencyTest` is the test the interviewer wants to see, and it was checked both ways: with caller-order locking it deadlocks and times out at 20 seconds, with id order it passes in 50 milliseconds. Do that experiment yourself once so you can describe it.

05-sql-transfer replaces the Docker/Postgres evening from the earlier plan: no install, the tests open two database sessions on two threads and script the interleaving step by step, so the lost update, the blocking `FOR UPDATE`, the deadlock the database detects and the stale optimistic writer are each one named test you can read. Two findings from running it with real threads are written into its README and worth repeating in the interview: a plain `UPDATE` takes a row lock, so opposite-direction transfers deadlock even with no explicit lock unless rows are written in id order; and optimistic locking thrashes on a hot row, which is why a ledger prefers `FOR UPDATE`.

7.5 Schedule

DAY	WORK
1	Clone, <code>./mvnw test</code> . 01-ledger run 1 (stories 1 to 3). Read sections 6.1 and 6.2
2	01-ledger run 2 with the idempotency extension. 02-load-balancer run 1. Sections 6.3 and 6.7
3	05-sql-transfer , all five stories. Write one page of notes on what you saw
4	01-ledger run 3 with history. 03-url-shortener . Sections 6.4 to 6.6, define every term out loud

DAY	WORK
5	<code>04-rate-limiter</code> . <code>02-load-balancer</code> run 2. Mock interview: a friend pastes the stories from the README into a chat window one at a time
6	<code>01-ledger</code> run 4, full 60-minute simulation including 20 minutes of Q&A from section 6. <code>06-ledger-http</code> if there is energy left
Day before	<code>01-ledger</code> run 5, light. Re-read sections 1 and 8. Sleep

8. Interview-day script

Opening (2 min). Camera on, IDE open on the template, notes file open. When the first story arrives: paste it into `notes.md`, read it back in one sentence, ask two questions ("Can amounts be negative? Should transfer to the same account be an error or a no-op?"), state the plan ("I'll model `Account` and `Ledger`, write a failing test for deposit, then implement").

Per story (8 to 10 min). Test first when it is cheap, implementation, run green, say what edge cases you covered and one you skipped and why. Ask "shall I move on?" Do not refactor mid-story; note it and come back if there is time.

Story 3 (concurrency). Before writing code say: "Two risks: check-then-act on the balance, and lock ordering across two accounts. I'll lock per account, acquire in id order, and prove it with a multi-threaded test." That sentence alone covers most of the checklist.

Q&A. Answer shape: definition, problem it solves, example from the code you just wrote, trade-off. If you do not know a term, say so and ask for a one-line hint rather than guessing; the Aug 2026 grad lost more by bluffing on DDD/CQRS than he would have by asking.

Closing. Ask one real question about the team's stack (jOOQ vs an ORM, how they test concurrency in CI). Thank them. Feedback arrives in a day or two.

Things that were marked down in reports and cost nothing to avoid: typing before clarifying, silence longer than 20 seconds, `double` for money, not running the tests, "memory addresses" instead of "buckets", forgetting the Java name of a method you just used (`contains`), and saying "I would use Spring for that" (there is no Spring in this stack).

9. Poland-specific notes

- Hub: Kraków. Hybrid, about 3 days a week in the office. Programme is 12 months, start early 2027 (January to June) or late 2027 (July to December). Full relocation package is offered for international hires, not needed for him.
- Polish candidates on 4programmers report the same account-transfer task since 2018 and that missing concurrency handling was a one-line rejection. One reported a second-stage call of "almost 2 hours, only database questions" with two non-native English interviewers.
- Interviewers are frequently non-native English speakers and are described as flat or "stone-faced". Ask them to repeat if unclear; it is not held against you. It is held against you if you code the wrong thing.
- Compensation reference from Polish job boards for mid/senior Java at Revolut Kraków: 20 to 30k PLN gross UoP. Graduate figures are not public; expect below that band.

10. Sources

- **Revolut blog: How to ace your Java interview at Revolut (March 2025)**
<https://www.revolut.com/blog/post/how-to-ace-your-java-interview-at-revolut/>
- **Revolut Tech on Medium: 5 Tips for your Live Coding Interview (the link in the invitation email)**
<https://medium.com/revolut/5-tips-for-your-live-coding-interview-c6d5b05adf97>
- **Revolut Careers: Graduate Programme 2027, Software Engineer (Java)**
<https://www.revolut.com/careers/position/graduate-programme-2027-software-engineer-java-3b79c804-b7e0-4b1a-9ef6-2fd69723dc7a/>
- **PracHub: Revolut New Grad Software Engineer, rejected after a concurrency deadlock (Aug 20 26)**
<https://prachub.com/interview-experiences/revolut-new-grad-software-engineer-interview-experience-rejected-after-a-concurrency-deadlock>
- **PracHub: Revolut Software Engineer concurrency and database interview (Sept 2026)**
<https://prachub.com/interview-experiences/revolut-software-engineer-concurrency-and-database-interview-experience-0163ac45cb>
- **Taro: Java Developer, Poland (Aug 2024)**
<https://www.jointaro.com/interviews/companies/revolut/experiences/software-engineer-java-developer-poland-august-1-2024-no-offer-negative-38007d04/>
- **Taro: Software Engineer, Poland, offer (July 2025)**
<https://www.jointaro.com/interviews/companies/revolut/experiences/software-engineer-poland-july-1-2025-accepted-offer-positive-adcc9513/>
- **Taro: Graduate Software Engineer, UK (Dec 2024)**
<https://www.jointaro.com/interviews/companies/revolut/experiences/graduate-software-engineer-united-kingdom-december-7-2024-no-offer-negative-5667b4b5/>
- **Taro: Java Backend Engineer, Porto**
<https://www.jointaro.com/interviews/companies/revolut/experiences/java-backend-engineer-porto-oporto-april-1-2021-no-offer-neutral-fcdb13ad/>
- **Reddit r/cscareerquestionsuk: Senior Java live coding expectations (Feb 2026)**
<https://www.reddit.com/r/cscareerquestionsuk/comments/1rgbu8z/>
- **Reddit r/cscareerquestionsuk: Stage 2 live coding in the last 6 months (June 2026)**
<https://www.reddit.com/r/cscareerquestionsuk/comments/1txw4pn/>
- **GitHub: virtyaluk/revolut_coding, Revolut live coding questions with solutions, confirmed current Dec 2025**
https://github.com/virtyaluk/revolut_coding
- **4programmers.net: Revolut Kraków (employer opinions and interview reports)**
https://4programmers.net/Forum/Opinie_o_pracodawcach/316909-revolut_krakow
- **Glassdoor: Revolut interview questions**
<https://www.glassdoor.com/Interview/Revolut-Interview-Questions-E1176471.htm>
- **Blind: Revolut interview posts**
<https://www.teamblind.com/company/Revolut/posts/revolut-interview>